

Game Dev Challenges and Programming

 The picture can't be displayed.

A book by ADSkeptic (Github)

Basic Concepts

Assets & Pipelining

The collective entirety of a game's playable "world" alternatively called **gameplay universe** by my wiser dictionary (or so I believe!), is ultimately made of some basic 3D or 2D "things", "stuff" and objects and a set of rules that define how they are to behave in the game.

Assets

Mesh

An actual "physical" object in the world.

Textures

Are 2D images used for different purposes. There are different variations of textures defining an equally necessary aspect of for it's mapping.

Noise

A layer containing procedurally generated values according to an Algorithm to be later used for creating texture patterns basically.

Albedo Map / Diffuse Map

Is the image itself that's usually meant to "coat" a mesh.

Height Map / Grayscale

A drawing using grayscale color range where the higher/extruding points are usually brighter.

Make sure the contrast is enough to depict the separation of details or you'll end up with a big blob.

Normal Map

Replaces RGB with surface info ("extrusion" and the like) in xyz.

Used for fake lighting details.

Mask Map / Control Map / Alpha Mask/

Opacity Mask / Roughness Mask / Gloss

Mask / Specular Mask

Controls where and how effects apply.

Materials

These are shader setups defining how to take textures and simulate. They are instructions for rendering data and are mainly concerned about how to render things on "surfaces".

Material Instances

Tweaked version of material still reliant on the base version. e.g: We have a Sky material then we create a morning one outta it

Cell

Game worlds are purposefully divided into chunks for manageability upon load.

These chunks are called Levels/Cells/Worlds in Game Engine system view.

From open world games to linear ones they all use this system but in open world ones typically the world loading should happen by more estimations and according to variables such as draw distance and player location to keep the open world from being loaded late.

Level/UMap

A saved definition of a noticeably large world section.

UWorld

A runtime simulation instance created from one or more umap defined levels.

The engine could run/bind to only one UWorld at a time.

Pipelining

Coordinate Systems

In computer graphics every shape is ultimately made of an organization of vertices. Organization happens thru systemic arrangement which itself in this case, is achieved through coordinate systems. Coordinate systems are conventions that define a **number of generic axes**, **their logical placement** (not their direction necessarily) using a **distance measuring unit** according to a **generic origin/starting point**. The extra stuff such as the direction of axis, the domains they cover (in which our shapes/vertices can exist), their established origin point implementations and etc may or may not be defined as a part of the coordinate system and are usually **implementation dependent coordinate conventions**. Consequently for each shape to be able to independently exist it would need to locally:

- Decide it's coordinate system, establish the conventions (origin, directions etc)
- Define it's own local axis, in relativism to which, it settles things (things being vertices in computer graphics).

The real challenge is that the **implementation dependent coordinate conventions** could demand different orders/settings according to what they are prioritizing.

(I know it sounds difficult to understand but before giving up look at the example it'll help you understand)

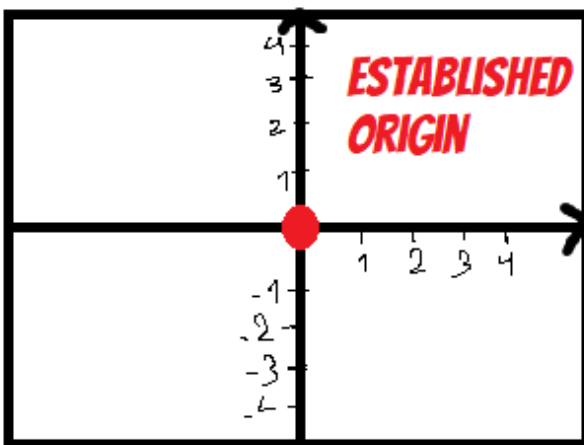
e.g: Normally on paper we consider the x,y coordinate system with x and y axes in which:

- x a horizontal and y a vertical axis.
- A single origin is set for both axis, by convention at the middle of the paper.
- A single fixed universal measuring unit is used.
- The direction of x left to right (values increase from 0 rightward, decrease from 0 leftward) and the direction of y increases bottom up (values increase since 0 upward and decrease since 0 downward).
- The domain of both axis is $\mathbb{R}$.

While this approach allows us to cover every logical aspect and side of our axis since the established origin and also makes mirroring a lot easier by providing us both sides of each axis, it requires some extra steps... that we might not need at times, especially when speed and performance is the priority for us,

UV Coordinate System

SPACE

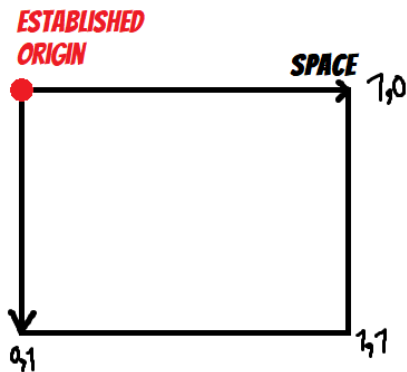


UV coordinate system, basically replaces the axis X and Y (in order) with axis U and V which it'll use for marking the distance of the vertices from an implementation dependent **established origin**, with a **relative distance measuring unit** within a **established domain**.

- In UV coordinate system the domain is implementation dependent.
- The distance measuring unit is relative to the object.
- The established origin(s) is also implementation dependent.

Typically though the following convention is implemented:

- There's a single origin for both U and V and it's at the topmost-leftmost point of the picture/2D plain.
- The direction of V is top-down (Values increase since 0 (origin) downwards), and the direction of U is rightward (Values increase since 0 (origin) rightward).



- The domain for both U and V is $D \leq 1$ with one representing “the end of the plane”.

The picture above depicts a pretty good example.

Shader Node Graphs

Abstractions for basic functions involved in shading.

Graphics Engine

Lights

Global Illumination

Light Maps (old solution)

RTGI (new solution)

Reflections

Generally there are 2 main reflection models:

- 1- Blinn Phong
- 2- Physically Based Rendering/Microfacet BRDF

- In the Blinn Phong model:

Final Color = Diffuse + Specular Highlight

- In the modern PBR instead we have:

Base Color, Metallic, Roughness

Texture Cubes (Blinn Phong)

Used static textures as reflections.

Screen Space Reflections

Dynamic reflections cast.

Stencil

Culling

Technical Art Concepts

Sky

Previously implemented inside a dome (sphere mesh) with materials defining what was rendered on it but new procedural methods have emerged in UE5 for per pixel rendering such as the atmosphere role.

Sky Material

The implementation of sky using material is done through:

- 1- Implementing a gradient: We manipulate the texture coordinates conventions to get it to give a direct linear multiplier to later be used for defining the strength of colors in gradient. (in the UDK example we did this using texture coordinate node, since in UDK the Domain is $[0,1]$ with the UV origin at the top left with V raising downward and all values until the very end, fractional, we multiply them to give them a simple direct

relationship and then get the radix complement using one minus to make the bias towards 0 go the opposite direction and redo the same process to lessen the bias.

- 2- Implementing the Colors based on Gradient: We use linear interpolate to get a mix according to the gradient by giving it as the alpha.

Achieving tiling

When we tile a texture , we are extending it's domain thru multiplying it as many times as the tiling.

Adhoc

Rendering

Optimization Aliasing

Reduction

Aliasing happens as the details get too small for the pixels to be rendered well enough. The reason in many older games this wasn't as big of a deal was with how

the engines handled 3D objects through decreasing the LOD / Using a decreased LOD version of objects after a distance back in the days of DL1 and such.

3D Modelling

Optimization Technique

Typically the 3D models use rather simple meshes for hair and then proceed to implement the hair thru textures transparency to give the illusion of empty pots in hair.

UDK (Older Approaches)

Upon running UDK it loads the core modules and Engine packages (some basic default template) upk files which are precompiled asset libraries mounted at runtime.

In UDK sky is a dome, like an actual 3D mesh as a dome.

Pro tip: Do not mistake a udk and a upk file. First is a map, the actual wrapper for the whole things arrangement, second is just a repository of assets.

Try saving your shit in a package with a different name than the map otherwise your entire repository of assets will just decide to disappear one day.

UDK Packages

UI Design

Apparently during the UDK days, there was no integrated UI Designer as a part of the kit itself such as in UE4 where Gfx or whatever it's name was allows for that. Instead UDK relied on it's support for Adobe Flash Professional's SWF format for the UI designs! Basically the dev had to buy that app as well and design the UI in it!

Material Editor

The material editor actually builds the GPU shader graph for materials.

It uses a chain of nodes to achieve it's results. This chains of nodes is made of:

- Nodes:basic building blcoks of material graphs.

Within the material editor we use **ctrl + click** to drag the nodes close and away to each other.

In order to connect nodes to one another we click on their pins. In order to cut connection among nodes we use **Alt + Click**

Nodes

Each node could be from:

- 1- The material function library:**
- 2- Material Expression:**

Either way the input pins of a node are on the right side, with the output pins on the left side.

IMPORTANT: Nodes sharing the same name and type will both become mirrors so watch it!

Material Output Nodes

These are the final interface between graph and the GPU shader result. Everything built in graph must ultimately be plugged into them. It defines the values the generated shader must compute.

- **Every material must have ONE single output node.**
- **The output node is always there by default with a pretty descriptive name like Material or PreviewMaterial_2.**
- **The output node does not have an output pin cause... it IS the output pin itself.**

The output node has the following input pins:

- 1- Diffuse: Takes color, constant3vector.
- 2- DiffusePower:
- 3- Emissive:
- 4- Specular: Strength of light reflection effect, takes a number.
- 5- SpecularPower: The amount of area of reflection divided.
- 6- Opacity:
- 7- OpacityMask:
- 8- Distortion:
- 9- TransmissionMask:

- 1- TransmissionColor:
- 2- Normal:
- 3- CustomLighting:
- 4- CustomLightingDiffuse:
- 5- AnisotropicDirection:
- 6- WorldPositionOffset:

Material Expressions

They are all the non output nodes that we add.

Constant3Vector

It represents a color.

TextureCoord

Component Mask

Takes a component ed value and treats it's first 4 component as RGBA, with Component Mask allowing you to mask off the components that you don't like using it. All components except the ticked ones will be masked off.

Linear Interpolate (Lerp)

Mixes pin B and A using Alpha as direct quotient for pin B and indirect quotient (through radix complement) for pin A,

Vector Parameter

Doesn't have or take any inputs. We set a color in it, outputs a single RGB or separate ones.

Scalar Parameter

This one represent a single scalar value, takes no input, just defines and outputs.

Panner

Takes inputs:

- Coordinates: (U,V)
- Time: N

And outputs: $(U + (\text{Speed } U * N))$, $(V + (\text{speed } V * N))$.

If the Time is not set it is set in real time, it could and usually is used for real time incrementation

Textures

Any average picture file is considered and supposed to be regarded as 2D texture by the engine. The PNG format is preferred and the goto one as other formats (such as JPG in our case) did cause some unusual behavior.

Texture Editor

Typically we define textures in the following resolutions

256x256 or 512x512 (Small Props)

Cans, books, etc.

512x512 or 1024x1024 (Medium Props)

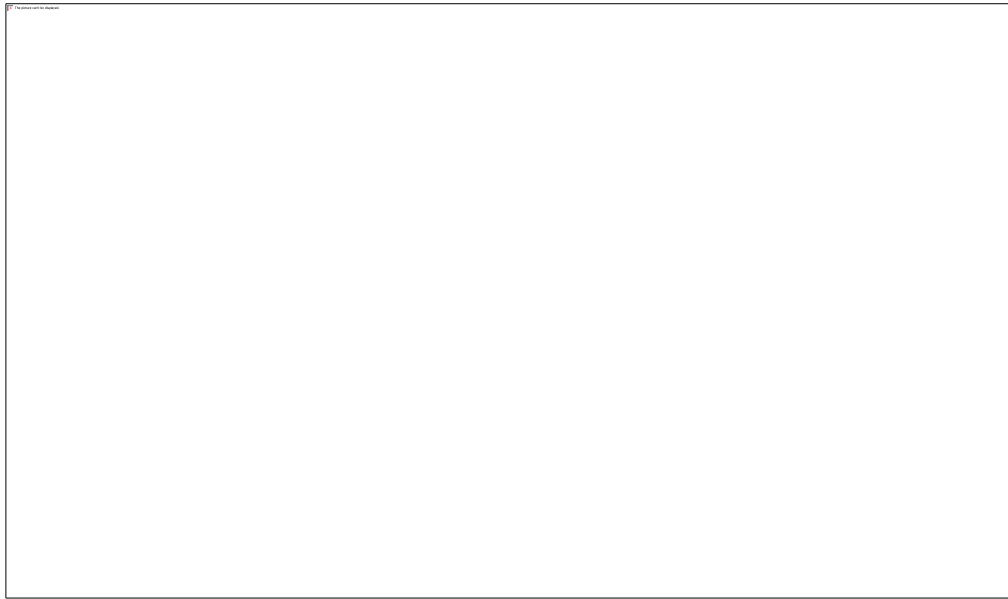
Crates, weapons, furniture.

1024x1024 or 2048x2048 (Hero Assets)

Player weapons, important characters.

Unreal Engine 5.7

Documentation



	Uses these menus to access common application actions, like saving, and creating
1 Menu Bar	new levels. You'll also find options for opening editor windows and tools that are useful for specific functions like debugging and more.
2 Main Toolbar	Contains shortcuts for some of the most common tools and editors in Unreal Engine, as well as shortcuts to enter Play mode (run your game inside Unreal Editor) and to deploy your project to other platforms.
3 Viewport Toolbar	Includes common tools used to manipulate objects in the level by moving, rotating, and scaling them, as well as snapping tools for moving objects along a grid, rotation angle, or scaling amount. It also includes perspective and

orthographic views along with debugging and visualization view mode and other settings you can use while working in the viewport.

4	Level Viewport	Displays the contents of your Level, such as Cameras, Actors, Static Meshes, and so on.
5	Outliner	Displays a hierarchical tree view of all content in your Level.
6	Details panel	Appears when you select an Actor. Displays various properties for that Actor, such as its Transform (position in the Level), Static Mesh, Material, and physics settings. This panel displays different settings depending on what you select in the Level Viewport.
7	Content Drawer	Opens the Content Drawer, a temporary Content Browser window, from which you can access all of the Assets in your Project.
8	Bottom Toolbar	Contains shortcuts to the Command Console, Output Log, and Derived Data functionality. Also displays source control status.

The **Main Toolbar** contains shortcuts to some of the most used tools and commands in Unreal Editor. It is divided into the following areas:



Number	Name	Description
1	Save button	Click this button to save the level that is currently open.

Number	Name	Description
2	Select the open level	Click this button to show the currently loaded level in the Content Browser recently used Content Browser, which can be the docked Content Drawer or the Content Browser panel.
3	Mode selection	Contains shortcuts for quickly switching between different modes to edit content. These modes also change the primary behavior of the Level Editor, which is designed for each mode specifically. You can click the links to go to their respective pages for more about a mode. • Selection • Landscape • Foliage • Mesh Paint • Modeling • Fracture • Brush • Animation
4	Content shortcuts	Contains shortcuts for adding and opening common types of content within the Level Editor. 1. The Create button can be used to choose from a list of common Assets. Used assets from the Content Browser to quickly add to your level. 2. The Blueprints button can be used to create and access blueprints.

Number	Name	Description
		3. The Cinematics button can be used to create and access a Level Sequence cinematic.
5	Play mode controls	Contains the shortcut buttons Play, Skip, Stop, and Eject for running your
6	Platforms menu	Contains a series of options you can use to configure, prepare, and deploy platforms, such as desktop platforms such as Windows, macOS and Linux. Some platforms, such as mobile and console, require additional setup and project. These can include their own software development kit (SDK) installation agreements (NDAs) to access them.

Viewport Toolbar

The Viewport Toolbar can be used to accomplish a number of tasks when building levels. It is located at the top of the Level Viewport, and its settings and tools are grouped into the following categories:



Number	Name	Description
1	Transform and Snapping tools	These are the options to swap between different Transform tools options.
2	Camera tools	These are the options to switch between Perspective and Orthographic camera movement speed.

Number	Name	Description
3	View Mode and Show Flag options	These are the options to pick between different view modes like and show flags related to the current viewport to hide and reveal Level Viewport.
4	Performance and Scalability tools	This dropdown can be used to reveal settings like Viewport Scalability Level.
5	Viewport options	These options can be used to change viewport related settings, like Level Viewport.

To learn more about this toolbar, read the [Viewport Toolbar](#) documentation.

Personal Understanding

Unreal Engine Routine

Project Creation

Upon running the unreal engine for the first time/setting a directory a **uproject** is made in that directory, marking the root of the unreal projects. Also generating all the necessary files/folders that and the structure of files to later be used in the process.

Game World Sections

Cell

Game worlds are purposefully divided into chunks for manageability upon load.

These chunks are called Levels/Cells/Worlds in Game Engine system view.

From open world games to linear ones they all use this system but in open world ones typically the world loading should happen by more estimations and according to variables such as draw distance and player location to keep the open world from being loaded late.

Level/UMap

A saved definition of a world section.

UWorld

A runtime simulation instance created from one or more **umap** defined levels.

A world is what's loaded in Memory.

A umap/level is what's residing on disk.

It's the home for all events to take place in.

Outliner

It represents and loads the engines engine specific filesystem accordingly. It shows the content of the currently loaded umap/level and does what sceneview did in unreal 3.

Content Browser

Instead of upk files in UE5 all the assets exist in their “Raw Form” without any more packaging involved than what the OS’s directory system offer.

Material Editor

Serves the same purpose as in UDK with some minor differences in UI.

Nodes

Each node could be from:

- 1- **The material function library:**
- 2- **Material Expression:**

Either way the input pins of a node are on the Left side, with the output pins on the Right side.

Material Output Nodes

These are the final interface between graph and the GPU shader result. Everything built in graph must ultimately be plugged into them. It defines the values the generated shader must compute.

- **Every material must have ONE single output node.**
- **The output node is always there by default with a pretty descriptive name like Material or PreviewMaterial_2.**
- **The output node does not have an output pin cause... it IS the output pin itself.**

It is worth mentioning that

- 1- Base Color: Takes color.
- 2- DiffusePower:
- 3- Emissive:
- 4- roughness:
- 5- Opacity:
- 6- OpacityMask:
- 7- Distortion:
- 8- TransmissionMask:
- 9- TransmissionColor:
- 10- Normal:
- 11- CustomLighting:

- 12- CustomLightingDiffuse:
- 13- AnisotropicDirection:
- 14- WorldPositionOffset:

Widgets

Are created content explorer and then used. It allows us to set a bunch of widgets using the details section on the right. Where we set “Anchors” and then in relativism to them set the element itself. Keep in mind everything seen in the widget’s preview is a mere preview within the size of the preview at the moment and WILL change in the time of running it.

Normal Details

The following Normal Details operations are available for widgets.

Anchor Operations

Sets the anchor and the position of widget in relativism to it.

Canvas Panel

A canvas panel is used as the root coordinate system like container for the rest of the stuff inside it.

Image

Is used to display a material/Texture.

List View

Force fits the items inside it as a vertical list (where no 2 items get to exist in the same line) and possibly adds scrolling if the size of the List View is too small to contain them all. It acts like a stack used for automatic list arrangement and positioning where custom alignment of the objects within the list is not a focus.

Vertical Box

Is used for when we want the objects within the box to show up as a list but want to align their positioning manually. It's List View for a predefined constant list.

Side Quest